

Lecture 2

A History of Natural Language Processing

Four paradigms, two winters, and what each transition actually cost



Seven dates that reorganised the field. Two of them were setbacks.

Numan Aslam

Where we left off, and where we go today

From Lecture 1

- Language is ambiguous, context-dependent, and has an endless frequency tail
- Six levels of linguistic analysis, from orthography to discourse
- Six task shapes that nearly every problem reduces to
- Two ways to build a system: an explicit pipeline, or an end-to-end model

Today's question

How did the field get from hand-written grammars to models trained on a substantial fraction of the written internet? And, more usefully: what forced each change, and what did the field give up each time it changed?

01

Origins

1948 to 1966. Information theory, the Turing test, the first translation machine, and the first collapse.

02

Symbolic era

1966 to late 1980s. Grammars, knowledge bases, and why hand-written rules could not scale.

03

Statistical turn

Late 1980s to 2000s. Corpora, annotation, probability, and feature engineering.

04

Neural and pretraining

2003 to now. Learned representations, the Transformer, and the shift to scale.

Why a computer scientist should care about this

1 The tools did not die

Regular expressions, edit distance, n-grams, hidden Markov models and TF-IDF all date from earlier paradigms and all remain in production today. Weeks 3 to 8 of this course are largely built from them.

2 The failures repeat

Every paradigm was, at its peak, believed to be nearly complete. Each time, the claims ran ahead of the evidence and funding followed the claims down. Recognising that pattern is a professional skill.

3 The vocabulary is historical

Terms like 'parsing', 'tagging', 'alignment', 'decoding' and 'language model' carry meanings fixed decades ago. Papers assume you know them.

4 The open problems are old

Coreference with world knowledge, pragmatics, low-resource languages and honest evaluation were hard in 1966 and are hard now. Knowing what has been tried saves you from retrying it.

Four paradigms, each absorbing the last

1948 – 1966

Origins

Information theory, the imitation game, the Weaver memorandum, Georgetown-IBM. Enormous optimism on very little evidence.

Ends with ALPAC and the first funding collapse.

1966 – 1988

Symbolic

Hand-written grammars, lexicons and logic. ELIZA, SHRDLU, LUNAR, expert systems, generative grammar.

Precise and interpretable, but brittle and unscalable.

1988 – 2012

Statistical

Learn from annotated corpora. HMMs, CRFs, IBM alignment models, n-gram language models, the Penn Treebank.

Robust to noise, but dependent on hand-designed features.

2013 – now

Neural, then pretrained

Learned representations. word2vec, seq2seq, attention, the Transformer, BERT and GPT, in-context learning.

General and powerful, but expensive and opaque.

Dates are approximate and the boundaries are contested. Treat them as centres of gravity, not sharp transitions: statistical methods existed in the 1970s, and rule-based components are still shipping today.

Origins: four ideas that started the field

1948

Shannon, A Mathematical Theory of Communication

Introduces entropy and models English as a Markov process, generating text from n-gram statistics.
Thttps://www.youtube.com/watch?v=R4OIXb9aTvQ

The statistical view of language arrives first, and is then ignored for forty years.

1949

Weaver, the Translation memorandum

Proposes treating translation as a decoding problem: a Russian text is English in a strange code.
Directly inspires the first machine translation programmes.

Frames MT as computation rather than scholarship.

1950

Turing, Computing Machinery and Intelligence

Proposes the imitation game, making conversational language ability the operational test of machine intelligence.

Sets language as the benchmark for AI, where it has largely remained.

1954

The Georgetown-IBM experiment

Around sixty Russian sentences translated into English using six grammar rules and a 250-word vocabulary. Widely reported as a breakthrough.

Researchers predicted translation would be solved in three to five years.

1966: ALPAC and the collapse of machine translation

Twelve years of optimism ended with a committee report that nobody in the field expected.

What the report found

- Machine translation was slower, less accurate, and more expensive than human translation
- No general-purpose working system existed after a decade of funding
- Output required so much post-editing that little was saved
- There was no shortage of human translators to justify the urgency

What it caused

- US government funding for machine translation was largely withdrawn
- The field fragmented; MT research continued mainly outside the United States
- Attention shifted from translation to understanding, which seemed more tractable
- A generation of researchers learned to be cautious in public about timelines

The lesson, which the field has had to relearn

The underlying research was not worthless, and much of it was later vindicated. What failed was the gap between a curated demonstration and a deployable system, and the willingness to let that gap go unstated. ALPAC also did something valuable: it insisted on measuring cost, accuracy and human effort rather than accepting selected examples. That insistence on evaluation is the direct ancestor of the benchmark culture we will criticise in Week 9.

The symbolic era: knowledge written by hand

ELIZA

1966, Weizenbaum

Pattern matching on keywords with scripted transformations. The DOCTOR script imitated a therapist by reflecting the user's own words back.

Users attributed understanding to a program with none. Weizenbaum was alarmed enough to write a book about it.

<https://www.youtube.com/watch?v=8jGpkdPO-1Y>

SHRDLU

1970, Winograd

Genuine understanding of commands about a simulated world of coloured blocks, including reference resolution and simple planning.

Impressive and entirely confined to the blocks world. It did not generalise, and could not.

<https://affinelayar.com/shrdlu/>

LUNAR

1971, Woods

Natural language question answering over a database of lunar rock samples, using an augmented transition network grammar.

Worked well on its domain, and demonstrated how much hand-crafting each new domain required.

Generative grammar

1957 onward, Chomsky

Syntax as a formal rule system. Supplies the formal-language hierarchy underpinning parsing theory.

Shaped how the field thought about structure, and pushed it away from statistics for decades.

Common thread: a human expert writes down the knowledge, and the machine applies it. Within its domain the result is precise, inspectable, and correct by construction.

Why hand-written rules could not scale

1 Knowledge acquisition bottleneck

Every rule must be elicited from an expert, written, tested and maintained. Coverage grows linearly with human effort while the phenomena grow faster than that.

2 Brittleness

A rule system either matches or does not. Unseen input produces no output rather than a degraded one, which is the wrong failure mode for language, where the unseen is guaranteed by Zipf's law.

3 Rule interaction

Rules conflict as the grammar grows. Past a few hundred, adding one rule routinely breaks others, and no one can hold the interactions in their head.

4 Ambiguity explosion

A broad-coverage grammar assigns a long sentence thousands of valid parses with no principled way to choose. Symbolic systems had no notion of which reading was more likely.

Point four is decisive: ambiguity is a ranking problem, and ranking requires probability.

The statistical turn: learning from corpora

Three enabling conditions

- Data. Machine-readable corpora became available, from the Brown Corpus of the 1960s to the annotated Penn Treebank in 1993.
- Compute. Workstations could finally estimate and store models over millions of parameters.
- Evaluation. Shared tasks and common test sets, notably the Message Understanding Conferences, made progress measurable and comparable.

The change in stance

Stop asking what a competent speaker knows. Start asking what a large sample of real language actually contains, and estimate the parameters of a model from it.

The question moves from 'is this sentence grammatical?' to 'how likely is this analysis, given everything I have seen?'

IBM alignment models

1990-93

Statistical machine translation learned from parallel text, using the noisy-channel framing Weaver had proposed forty years earlier.

Penn Treebank

1993

A million words of text annotated with part-of-speech tags and syntactic structure. Made supervised parsing possible and set the template for annotated resources.

Hidden Markov models

1970s-90s

Sequence labelling as probabilistic inference. The dominant approach to part-of-speech tagging and speech recognition for two decades.

“Every time I fire a linguist...”

“Every time I fire a linguist, the performance of the speech recognizer goes up.” — attributed to Frederick Jelinek, IBM, mid-1980s

What it got right

- Hand-written linguistic rules really were being outperformed by estimated parameters
- Systems that learn degrade gracefully on unseen input; rule systems do not
- Measurable, comparable error rates beat expert intuition as a guide to progress

What it got wrong

- Linguistic knowledge did not become useless; it migrated into annotation schemes, evaluation design and error analysis
- Someone has to decide what to label and what counts as correct, and that is a linguistic judgement
- The people who can build a test set that breaks your model are the ones who know what a garden-path sentence is

Why this still matters to you

The quote is often repeated as proof that domain expertise is obsolete. The actual record is narrower: hand-coding rules lost, and hand-designing what to measure did not. Every time a modern benchmark turns out to be measuring the wrong thing, it is because nobody with domain knowledge was in the room when it was designed.

The neural shift: features stop being designed

2003

Bengio et al., a neural probabilistic language model

Words become learned vectors and a network predicts the next word. Solves n-gram sparsity by generalising across similar contexts. Largely ignored at the time: too slow for the hardware.

2008-11

Collobert & Weston, NLP almost from scratch

One network trained on several tasks at once, with no hand-designed features. Competitive results on tagging and named entities, which made the argument concrete.

2013

Mikolov et al., word2vec

Cheap, scalable word embeddings from raw text. The moment distributed representations became standard equipment rather than a research curiosity.

2014-15

Sequence to sequence, then attention

Sutskever et al. map a sequence to a sequence with two recurrent networks; Bahdanau et al. remove the fixed-size bottleneck by letting the decoder attend over all encoder states.

Transformers, pretraining, and the turn to scale

2017**The Transformer**

Vaswani et al. discard recurrence and keep only attention. Every position is computed in parallel, so training on very large corpora becomes practical for the first time.

2018**ELMo, BERT, GPT**

Pretrain one model on raw text with a self-supervised objective, then adapt it to many tasks. Task-specific architectures largely disappear within two years.

2020**In-context learning**

Brown et al. show that a sufficiently large model performs new tasks from examples in the prompt, with no weight update. The interface to the model becomes natural language.

2022-**Alignment and adaptation**

Instruction tuning and learning from human preferences make models usable by non-specialists. Parameter-efficient methods make adaptation affordable again.

What changed underneath: the bottleneck moved again

In the symbolic era the bottleneck was expert time. In the statistical era it was annotation. Self-supervision removed both: the label is already in the text, so the constraint becomes raw data and compute. That is why progress since 2018 has been driven as much by scale as by architecture, and why the field now has an access problem it did not have before.

What actually caused each transition

PARADIGM	SCARCE RESOURCE	WHAT MADE IT POSSIBLE	WHAT IT GAINED	WHAT IT GAVE UP
Symbolic	Expert time	Formal grammars, logic, early compilers	Precision and transparency	Coverage and robustness
Statistical	Annotated data	Corpora, shared tasks, cheap workstations	Robustness and measurable progress	Interpretability of individual decisions
Neural	Raw data and GPUs	Backpropagation at scale, embeddings	Generalisation across similar inputs	Explicit linguistic structure
Pretrained	Compute	Self-supervision, parallel attention	One model for many tasks	Auditability, and open participation

Read the last column carefully. Nothing in this table is a free upgrade.

Four patterns that repeat

1 Demonstration mistaken for capability

Georgetown-IBM in 1954, SHRDLU in 1970. A curated example on a narrow domain gets reported as a general solution, and the correction arrives years later.

Ask what the system does on input nobody chose for it.

2 Claims outrun evidence, then funding follows

ALPAC in 1966 and the expert-systems collapse around 1987. The research was rarely worthless; the promises were.

Distinguish a result from a roadmap.

3 Ideas wait for their conditions

Shannon's n-grams in 1948, Weaver's noisy channel in 1949, Bengio's neural language model in 2003. Each was correct and unusable until data and hardware caught up.

An old idea is not a dead one.

4 Evaluation drives progress, then distorts it

Shared tasks made the statistical era measurable. Benchmarks now saturate, leak into training data, and get optimised for their own sake.

Whoever defines the metric defines the field's direction.

Nothing was replaced. Everything was absorbed.

From the symbolic era

- Regular expressions, still in every production pipeline
- Finite-state methods for morphology and normalization
- Rule-based guardrails around generative systems
- The formal vocabulary of grammar and parsing

Covered in Week 3

From the statistical era

- n-gram language models and perplexity
- TF-IDF and BM25, still the retrieval baseline to beat
- Naive Bayes, logistic regression and SVM for classification
- Annotated corpora, shared tasks, and held-out evaluation

Covered in Weeks 4-8

From the early neural era

- Word embeddings and cosine similarity
- Encoder-decoder framing for sequence problems
- Attention, which became the whole architecture
- The habit of learning representations rather than designing them

Covered in Weeks 4, 10-12

For a narrow task with clean in-domain data and a latency budget, a 1995 method still frequently wins.

Key dates worth remembering

1948

Shannon models English with n-grams

1949

Weaver's translation memorandum

1950

Turing proposes the imitation game

1954

Georgetown-IBM translation demonstration

1957

Chomsky, Syntactic Structures

1964

Brown Corpus completed

1966

ALPAC report; ELIZA

1970

SHRDLU

1971

LUNAR question answering

1990-93

IBM statistical translation models

1993

Penn Treebank released

2003

Bengio, neural probabilistic language model

2011

Collobert et al., NLP almost from scratch

2013

word2vec

2014-15

Sequence to sequence, then attention

2017

The Transformer

2018

ELMo, BERT, GPT

2020-22

In-context learning, instruction tuning

Five things to take away

1 The field has moved through four paradigms

Symbolic, statistical, neural, pretrained. Each became dominant when a bottleneck moved, not when an idea was first had.

2 Each transition traded something away

Coverage for precision, interpretability for robustness, linguistic structure for generalisation, auditability for scale.

3 Failure came from overclaiming, not from bad research

ALPAC did not find that the science was worthless. It found that the promises were not met and had not been measured.

4 Old methods did not die

Regular expressions, n-grams, TF-IDF and linear classifiers are all still deployed, and most of this course's first half is built from them.

5 Evaluation is the thread through all of it

Every paradigm advanced when it became measurable, and stalled when the measurements stopped meaning anything.

Lecture 3: Setting up the NLP environment

What we will do

- Python version management, and why the system Python is not the one to use
- Virtual environments with conda or venv, and why you need one per project
- Installing and verifying NLTK, spaCy, scikit-learn and Gensim
- Jupyter, and the habits that keep notebooks reproducible
- git basics and the repository you will use for assignments

Bring to the session

- A laptop with Python 3.11 or later already installed
- A verified Kaggle account, so the free GPU quota is available
- A GitHub account
- Roughly 5 GB of free disk space for packages and model files

Reading for this lecture

Jurafsky & Martin, Speech and Language Processing (3rd edition draft), the historical notes at the end of Chapters 1 and 2. For anyone who wants the primary sources: Turing (1950) in Mind; the ALPAC report (1966), which is short and readable; and Weizenbaum's Computer Power and Human Reason (1976) for the ethics of the ELIZA effect, which we return to in Week 15.